Gatekeeper Design Document

Gatekeeper: Design Document

1. Summary

Gatekeeper is an RL environment for access-policy repair. The agent is given a world model, a broken policy and a natural-language policy specification. It uses a bounded tool interface to edit the policy, then submits it. A trusted verifier evaluates the submitted policy over a hidden, exhaustive request matrix. The core design question is not "can a model call tools?" but "can a model reason about authorization semantics without overfitting visible examples or leaking the answer key?"

The environment is deliberately model-agnostic: the harness speaks the OpenAI chat-completions tool-call protocol, with a lenient fallback for models that emit JSON in text. No model-specific prompting or fine-tuning is required.

2. Goals and non-goals

Goals

- A self-contained, deterministic task suite with easy, medium and hard tiers.
- A hidden answer key and exhaustive scoring path that cannot be inspected by the agent.
- Realistic authorization semantics: default deny, deny-overrides-allow, conditions, exceptions, nested groups and wildcards.
- Strong baseline gates that catch trivial or broken environments.
- A one-command Docker Compose workflow.
- Useful trajectories and baseline artifacts for judging and debugging.
- Clear failure modes: invalid config, crash, gate failure, model failure, step cap and wall-clock cap.

Non-goals

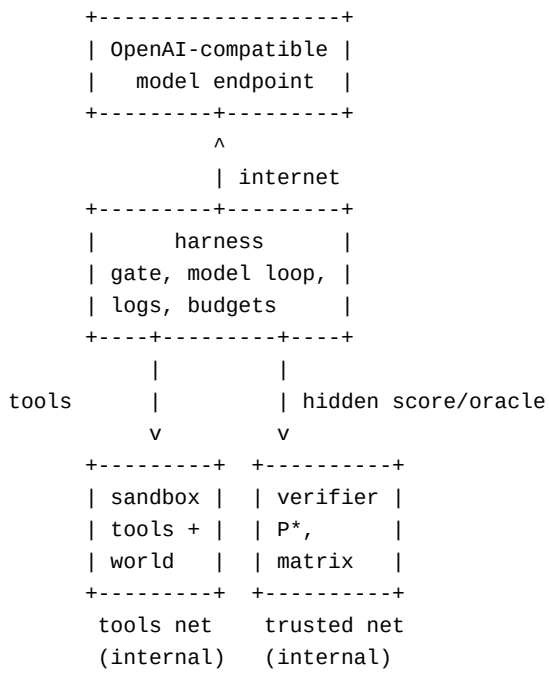
- General-purpose IAM administration or production policy deployment.
- Executing arbitrary code in the sandbox.
- Hard guarantees against a malicious harness operator. The harness is trusted; the untrusted boundary is the model and its tool calls.
- Training a model. The project supplies the environment and evaluation path.

3. Threat model

```
| Threat | Mitigation |
| Agent reads P*, mutations or the matrix | Only the verifier has them. Sandbox receives a sanitized
bundle. Verifier and sandbox are on separate internal networks. |
| Agent hardcodes visible examples | Full matrix scoring; visible examples are a small,
non-representative sample. |
| Agent submits allow-all or deny-all | Protected allow-all and restricted-deny baselines make both
score approximately zero; gate checks both. |
```

Agent submits the unchanged broken policy	No-op baseline gate and E_start denominator make a no-op score zero.
Prompt injection via APPEND_FILE	The file is explicit, audited, copied into the run log and confined to inputs/; it does not change tools or scoring.
Malformed or hostile tool calls	JSON object validation, Pydantic models, reference validation, 4K output cap, 10 calls/reply, 60s tool timeout.
Sandbox reaches the internet or verifier	sandbox is on an internal network only; verifier is on a separate internal network; only harness has egress.
Resource exhaustion	pids_limit, mem_limit, read-only filesystems for sandbox/verifier, cap_drop: ALL, no-new-privileges, step/time caps.
Answer leakage through logs	Gate logs contain scripted baseline actions only in the trusted harness log; the sandbox has no log mount or verifier route. Episode logs contain model-visible messages, tool output and state hashes, never the hidden matrix or answer key.
Non-determinism	All generation and sampling is seeded; canonical state hashes.

4. Architecture



There are three images. Docker Compose runs three services:

- sandbox - only on tools, which is internal. It has no route to the verifier or the internet.
- verifier - only on trusted, which is internal.
- harness - on tools, trusted and internet. It is the only process that can reach the model API.

The split prevents the sandbox from calling the verifier even if an input-validation bug existed. The harness must be trusted because it orchestrates the gate and the model; the model itself never receives verifier endpoints or keys.

5. Data model

World

A World contains:

- users: dict[str, User] - users have IDs and optional metadata.
- groups: dict[str, Group] - group members are user selectors or nested group selectors (user:<id>, group:<id>).
- resources: dict[str, Resource] - resources have IDs and tags. Tags use the tag:<name> selector form.
- actions: tuple[str, ...] - read, write, delete, share.

Policy

A Policy is an ordered collection of Statement objects. Order is not semantically meaningful; the evaluator sorts by statement ID and uses the first matching statement as the deciding statement. Statement fields:

- id - unique within the policy, 1-128 characters.
- effect - allow or deny.
- principals - *, user:<id> or group:<id>; up to 64.
- actions - * or concrete actions; up to 8.
- resources - *, resource:<id> or tag:<tag>; up to 64.
- condition - optional dict[str, str]; all keys must match; up to 16.
- exceptions - principal selectors exempted from the statement.

Canonical state

state_hash(world, policy) serializes the world and policy with sorted keys, sorted principals/actions/resources/exceptions, sorted condition keys and sorted statements by ID. The hash is a stable 8-hex-character prefix of SHA-256. Env.reset returns it and the harness writes it to the logs; it is not exposed as a tool. It is used to verify that the sandbox and verifier agree on the reset state.

6. Evaluator semantics

For a request (user, action, resource, context):

1. Start with the default decision deny.
2. Scan every deny statement in ascending ID order. A statement matches when:
   - the user matches at least one principal selector (direct user or effective group membership), and is not covered by an exception;
   - the action matches at least one action selector;
   - the resource matches at least one resource selector (direct ID or tag);
   - every condition key/value matches the request context exactly.
3. If any deny matches, the first matching deny decides. This makes deny-overrides-allow explicit and independent of statement insertion order.
4. Otherwise scan allow statements in ascending ID order. The first matching allow decides.
5. If no statement matches, the request is denied by default.

The evaluator compiles selectors into sets/frozensets and memoizes group expansion. It is deterministic and was originally checked against a naive

reference implementation on 400,000 random requests.

7. Task generation

7.1 World generation

The generator creates a seeded world with the configured number of users, groups and resources. Users belong to one or more groups; groups can contain nested groups. Resources receive tags, including a reserved scope:restricted tag used by the protected deny baseline.

7.2 Correct policy construction

The correct policy is deny-list-shaped:

1. A protected allow-all statement s001.
2. A protected deny-all-actions statement s002 on tag:scope:restricted.
3. Feature statements that deny risky actions on safe resources under specific conditions, groups or exceptions.
4. Random deny statements that exercise nested groups, tags, actions and conditions.

This shape guarantees that:

- allow-all is wrong (because of the restricted deny);
- deny-all is wrong (because of the allow-all baseline);
- an empty policy is wrong;
- the task requires understanding both broad and narrow rules.

s001 and s002 are protected: the mutation engine never edits, drops or flips them. Feature statements are the preferred mutation targets. Random statements provide fallback targets and additional semantic richness.

7.3 Visibility and sanitization

GeneratedTask.sanitized_bundle() returns exactly:

```
{
  "task_id", "tier", "seed",
  "world", "broken_policy", "requirements",
  "visible_examples", "state_hash",
}
```

The bundle contains no correct_policy, reference_decisions, reference_sample, mutations, matrix, e_start, sample_indices, shadow_indices or oracle_actions. A test recursively checks this property.

7.4 Mutations

The task config lists mutation classes. The generator applies each one to a statement that actually supports it and measures the impact on a fixed sample of the hidden matrix. It rejects a mutation if the sampled impact is below max(1, sample_size // 1000) and retries with other targets. The dedicated feature target is tried first; other compatible statements are fallbacks.

Destructive mutations (drop_statement, remove_deny) are ordered last so earlier mutations have stable targets.

The six classes:

```
| Class | Meaning |
| drop_statement | Remove a statement. |
| widen_wildcard | Replace a selector with *. |
| remove_condition | Delete a condition. |
| remove_deny | Remove a deny statement rather than merely editing it. |
| change_group | Point a group selector at another group. |
| flip_effect | Turn allow into deny or vice versa. |
```

7.5 Sampling and E_start

The full matrix is users x actions x resources x contexts. The generator computes full reference decisions with the correct policy, then computes the broken policy's full decisions and E_start:

```
E_start = 3 * false_allows_broken + false_denies_broken
```

It rejects tasks where E_start < max(20, total // 500). It also computes a seeded shadow sample and a separate sample for mutation-impact checks, so debugging is fast without changing the final score.

7.6 Requirements and visible examples

render_requirements renders the correct policy into deterministic natural language. It includes every group, statement, condition key, exception and the default-deny rule. It explicitly warns that some checked names and condition keys are not mentioned, encouraging directory inspection rather than memorization.

Visible examples are a small balanced sample of the hidden matrix (5-9 per task). They are useful for debugging but insufficient for a high score; a test constructs a policy that matches every visible example exactly and confirms it still scores below 0.95.

8. Scoring and gate

Full score

```
weighted_errors = 3 * false_allows + 1 * false_denies
score = max(0, 1 - weighted_errors / E_start)
```

- false_allow: the candidate allows a request the correct policy denies.
- false_deny: the candidate denies a request the correct policy allows.
- The score is 1.0 only for an equivalent policy over the full matrix.
- The score is clipped at 0, so making the policy worse than the broken policy is not rewarded.

The asymmetric weights encode the security preference: over-permission is worse than over-restriction. E_start normalizes difficulty: a task with more starting errors is not automatically easier or harder.

Shadow score

After each model turn, the harness asks the verifier to score the current policy on a fixed seeded sample. The sample has its own broken-policy error norm, so the correct policy scores 1.0 and the submitted broken policy scores 0.0 on that sample. This is cheap and gives a progress signal. It is not used to decide success. A model can see the shadow score in the episode log, but the final score is always the full-matrix score.

Gate thresholds

```
| Baseline | Requirement |
| Oracle | score >= 0.95 |
| No-op (broken policy) | score <= 0.05 |
| Allow-all | score <= 0.001 |
| Deny-all | score <= 0.001 |
```

The gate runs before the model on every selected task. The reference and no-op baselines are harness/agents.py:oracle_actions and noop_actions; the gate executes both through the same sandbox and verifier path as the model. Thin CLI wrappers in scripts/reference_solution.py and scripts/noop_agent.py expose the same paths for manual runs. A gate failure exits 2 and prevents the model call. This is the main defense against an environment that is trivially passable or accidentally denies/allow everything.

9. Environment and tools

Environment is initialized from the sanitized bundle. It validates that the supplied state_hash matches state_hash(world, broken_policy) before exposing state. It owns:

- world, initial_policy, current_policy, requirements, visible_examples;
- history for undo;
- submitted;
- a compiled evaluator.

Tool responses are clipped to 4,000 characters. put_policy validates the statement with Pydantic, checks that referenced users/groups/resources/tags/actions exist, and replaces by ID or appends. It does not allow a policy to grow past 200 statements. submit freezes the sandbox; later edits return an error.

The sandbox does not know P*, mutations or the correct decisions. Its evaluator is only for simulate and check_examples, both of which operate on the current candidate policy.

10. Harness

Loop

1. Wait for sandbox and verifier health.
2. Run the gate.
3. Reset the selected task and construct the system/user messages.

4. For each step:
   - enforce wall-clock and step limits;
   - call the model with native tools;
   - parse native tool calls, legacy function_call, or JSON-in-text calls;
   - enforce at most 10 calls per reply;
   - execute tools through the sandbox;
   - append tool results;
   - request a shadow score;
   - trim old assistant/tool units if the context budget is exceeded;
   - stop on submit.
5. Request the final full score.
6. Classify termination as success, failure, step_cap, time_cap or
   crash.

Parsing and robustness

The parser accepts:

- OpenAI native tool_calls (including SDK objects and dicts);
- legacy function_call;
- JSON objects/arrays embedded in text, optionally fenced;
- {"tool": ..., "args": ...} and {"name": ..., "arguments": ...} shapes.

Malformed JSON is not fatal: the call is marked with a parse error, the model receives an explanatory tool result, and the episode continues. Unknown tools and over-limit calls receive explicit errors. If no tool call is found, the model gets a short hint and the step is counted.

Budgets and context

Defaults: 40 steps, 600s wall clock, 10 tool calls/reply, 4K tool output, 60s tool timeout, 120s verifier score timeout, 144K characters of context. The context trimmer keeps the system message and first user message, then drops oldest assistant/tool units as complete units so a tool result is never separated from its call.

Suspicion telemetry

The harness records tool-call count, unknown tools, parse failures, path-like arguments, allow-all edits, repeated calls, simulate calls, submit calls, no-tool replies and maximum context ratio. This is diagnostic evidence, not a score input; a legitimate policy can add an allow-all baseline statement.

Structured telemetry

episode.jsonl contains one canonical step record per model reply. Each record has timestamp, agent, task, step, seed, model_input, model_output_raw, parsed_actions, tool_result, errors, state, reward, tokens and latency_ms. state is a hash plus statement count, not a full policy snapshot. reward contains the sampled shadow score and its components; the final episode_end record contains the full verifier result with a total reward alias. gate.jsonl uses the same canonical shape for

oracle, no-op, allow-all and deny-all actions, with agent set to the baseline name. These records are written by the trusted harness and are not mounted into the sandbox.

11. Determinism and reproducibility

- Task generation uses random.Random(seed) for world, feature selection, mutation targets, sampling and examples.
- The verifier caches tasks by (task_id, seed) under a lock.
- All decisions are deterministic by statement ID and canonical selector order.
- State hashes are canonical.
- The gate and episode logs include the seed, model name, step count and final score; the run summary records the append hash.
- A 1,000-seed generation soak on easy_01 completed with zero failures and zero determinism failures (mean 0.158s per seed, p95 0.163s); additional seeds were checked across medium and hard tasks. The all-task gate covers seed 0.

12. Testing and evidence

The repository includes 77 tests:

- evaluator unit and differential tests;
- models/canonicalization tests;
- generator invariants, determinism, sanitization and overfit resistance;
- verifier service and scoring tests;
- sandbox tool tests and seeded Env.reset tests;
- HTTP service and harness integration tests;
- parser, append, limits, fake model and one-command run tests;
- gate pass test.

Local evidence:

- All twelve tasks passed the four-baseline gate at seed 0 in about 15.7 seconds, including generation and scoring.
- Standalone generation soaks: easy_01 1,000 seeds with zero failures; easy_02 50; med_01 20; med_02/med_03/med_04 10 each; hard tasks 2-3 seeds each. No failures after the mutation fallback was widened.
- Easy generation is about 0.14-0.21s; hard generation is roughly 8-20s.
- The offline solver solves med_01 in three batched turns with score 1.0.
- The offline no-op, allow-all and deny-all clients all score 0.0.

Docker Compose was executed on the development machine on 2026-10-25: all three images built, the three services started, the med_01 gate passed (oracle 1.000, no-op/allow-all/deny-all 0.000), and the offline solver finished with score 1.0 in three steps. A clean --no-cache three-image build took 21.8s; the Docker all-task gate took 15.7s.

The same one-command contract was exercised with gpt-6-luna:

- med_01: success, score 1.0, 5 steps, 14.2s;
- easy_01: success, score 1.0, 5 steps, 18.1s;
- controlled no-op append on med_01: failure, score 0.0, 1 step, 2.1s.

The full logs are in test-run-logs/; the real success and failure trajectories

are also copied into sample-trajectories/.

13. Risks and tradeoffs

- Generation cost. Full-matrix scoring for hard tasks and candidate mutation trials is expensive. The generator uses a fixed sample for mutation impact and a cache keyed by (task_id, seed). Hard tasks took roughly 8-20 seconds on first generation.
- Deny-list shape. The protected allow-all baseline makes the correct policy less like a typical least-privilege policy. It is a deliberate anti-gaming tradeoff that guarantees allow-all and deny-all both fail.
- Harness trust. The harness has model and oracle access. A malicious operator could leak answers; the design protects against model actions, not a malicious host.
- Mock solver. The offline solver consumes oracle actions. It is useful for the gate/demo but is not evidence of model reasoning.
- No code execution. The sandbox tools are declarative. This reduces risk but also limits the environment to policy-repair tasks.

14. Future work

- Run a broader real-model sweep across models, temperatures, tasks and seeds, and report success/failure rates.
- Extend the generator to attribute-level policies and policy versioning.
- Add a differential fuzzer that compares the compiled evaluator with a naive interpreter on random policies and requests.
- Expand the generation soak across every task and publish timing percentiles.
- Produce a short demo video.
- Add an optional model-side defender that detects prompt injection in APPEND_FILE.